



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра алгоритмических языков

Пучкова Екатерина Михайловна

**Автоматическая оценка
удобочитаемости русскоязычных текстов**

Выпускная квалификационная работа

Научный руководитель:
к.ф.-м.н., доцент
Головин И.Г.

Москва, 2024

Аннотация

Проведен анализ основных подходов и систем автоматической оценки удобочитаемости. Предложен датасет, составленный из методических материалов для школы. Описана разработанная автором программа для оценки удобочитаемости русскоязычных текстов, превосходящая существующие аналоги в проблемной области оценки удобочитаемости школьных методических материалов.

Содержание

1	Введение	5
2	Постановка задачи	7
3	Обзор существующих работ	8
4	Теоретическая часть	12
4.1	Классические метрики удобочитаемости	12
4.2	Методы предварительной обработки данных	13
4.2.1	Стандартная нормализация признаков	13
4.2.2	Нормализация приведением к диапазону	14
4.3	Критерии отбора признаков	14
4.3.1	Критерий отбора признаков по величине коэффициента в формуле линейной регрессии	14
4.3.2	Отбор признаков по коэффициенту корреляции	15
4.4	Обзор алгоритмов многоклассовой классификации	16
4.4.1	Метод k ближайших соседей	16
4.4.2	Деревья решений	16
4.4.3	Наивный байесовский классификатор	18
4.4.4	Логистическая регрессия	20
4.5	Метрики оценки качества	21
4.5.1	Метрика ассигасу	21
5	Практическая часть	23
5.1	Предложенный подход к оценке удобочитаемости	23
5.2	Разработка датасета	24
5.3	Подсчет точности формул И.В.Бегтина	25
5.4	Подготовка данных для обучения и оценки качества моделей	27
5.5	Эксперименты	27
5.6	Программа автоматической оценки удобочитаемости	31
6	Заключение	34

1 Введение

Задача оценки сложности текста является одной из актуальных задач в наши дни. Мы сталкиваемся с данной задачей при проверке сложности дидактический материалов, проверке сложности экзаменационных тестов, проверке понятности нормативно-правовых актов, проверке уровня понятности деловой документации.

В связи с тем, что многие процессы в обществе автоматизированы, возникает желание автоматизировать процесс оценки сложности.

В данной работе вместо интуитивно понятного термина сложности оценивается величина удобочитаемости/читабельности текста. В работе [1] "читабельность" определяется, как характеристика текста, основанная на количественных параметрах, наиболее частыми по встречаемости из которых, являются количество слов в предложении, количество знаков или слогов в слове, количество слов, состоящих из трёх и более слогов и другие. Таким образом, можно понять, что термин "удобочитаемость" связывает количественные характеристики текста, которые легко поддаются подсчету на компьютере и значение его абстрактной сложности. Отсюда следует, что удобочитаемость это и есть та самая величина, значение которой следует оценивать при попытке автоматизировать процесс определения сложности текста.

Хорошей отправной точкой для изучения проблемы удобочитаемости русскоязычных текстов является проект Plain Russian Language[2], автор которого - И.В.Бегтин. Формулы, предложенные в проекте можно использовать в качестве базовой линии при оценке удобочитаемости. Однако точность формул по метрике ассигасу достаточно низка. Работа К.Ю.Дружкина[3] развивает идеи проекта И.В.Бегтина и использует некоторые методы машинного обучения, например, логистическую регрессию и метод опорных векторов для получения более точных результатов. Основная идея настоящей работы также состоит в использовании алгоритмов классификации для обучения "с учителем".

Подход, который реализуется в рамках работы, основан на исследовании применения нескольких известных алгоритмов классификации, описанных в главе 4. Построено 1600 вариантов экспериментальных моделей на основе перебора алгоритмов классификации, способов отбора признаков, способов нормализации признаков, количества признаков, используемых в модели. Множество из 50000 признаков выбрано на основе набора признаков, собранных в работе К.Ю.Дружкина[3]. Каждая из моделей была обучена на предварительно подготовленном наборе данных, в который вошли тексты

из методических материалов для школьников, студентов, а также тексты повышенной сложности.

По итогам проведенного исследования была разработана программа оценки уобочитаемости, которая использует наилучший по качеству вариант модели.

2 Постановка задачи

Задачей настоящей работы является разработка программы автоматической оценки удобочитаемости текстов на основе алгоритмов классификации с учителем. Основные этапы решения задачи:

1. Проанализировать существующие подходы к решению задачи и соответствующие им модели и алгоритмы
2. Разработать набор данных для проверки моделей оценки удобочитаемости текстов
3. Выбрать множество методов и моделей классификации для последующей оценки
4. Оценить качество работы моделей на разработанном наборе данных, сравнить его с существующими аналогами и выбрать наилучшую модель и набор ее параметров
5. Разработать программу автоматической оценки удобочитаемости, основанную на выбранном варианте

3 Обзор существующих работ

Одним из способов оценивания сложности является использование двух индексов: семантического и синтаксического. Первый заключается в оценке «незнакомости» слова, которая проверяется при помощи частотности слов в корпусе, а второй оценивает среднюю длину предложения. Данный подход осуществлен в анализаторе Lexile, запущенном на платформе Lexile Platform[5], которая активно используется в англоязычных странах для подбора учебников, тестовых материалов и развивающих книг для различных категорий читателей.

Также интерес представляет анализатор DRP, который расшифровывается как Degrees of Reading Power, что означает степень читательской способности, разрабатывавшийся для платформы Questar[6]. В нем были использованы формулы сложности Дж.Бормута с четырьмя предикаторами, в которые входили длина слова и предложения, количество знаков препинания и «узнаваемость» слова. Сложность текста в данном анализаторе ранжировалась по непрерывной шкале от 0 до 100, что означает, что более высокие значения маркируют более сложные тексты.

Среди анализаторов также встречаются поисковые роботы(web crawlers), предназначенные для поиска веб-текстов определенной тематики и уровней сложности. Одним из наиболее известных являются REAP[7] и TExtEvaluator[8]. Первый из них основан на оценке сложности отдельных слов и не оценивает более сложные семантические параметры. Его ключевым компонентом является расширенная модель поиска, находящая документы, имеющие определенную тему текста, тип синтаксических структур, словарный запас. REAP[7] использует тематический классификатор SVM(Support Vector Machine). Поиск выполняется на коллекции автоматически собранных из интернета документах, каждый из которых аннотирован и прошел процедуру метаразметки. Таким образом, REAP[7] является первым анализатором, в котором используется подход к оценке сложности методами машинного обучения.

Также можно ознакомиться с работой автоматизированного анализатора Coh-Metrix, предназначенного для оценки сложности и тестирования когнитивных моделей восприятия. Анализатор создавался на корпусе TASA[9], репрезентативность которого обеспечена объемом, а также тем, что он содержит примерно такое же количество текстов, которое среднестатистический студент колледжа прочел за всю свою жизнь[4]. Анализатор осуществляет глубокую обработку текста на лексическом и синтаксическом

уровнях, определяет связность текста и дискурсивные параметры текста, эксплицитность ситуационной модели. В основу разработки Coh-Metrix положена пятиуровневая когнитивная модель восприятия текста.

Для восприятия представленной в тексте информации реципиент должен иметь способность понимать отдельные слова, извлекать информацию из долговременной памяти, использовать синтаксические знания для синтеза пропозиций и макропропозиций, применять эксплицитные и имплицитные коннекторы для установления связей между предложениями и частями текста, а также использовать фоновые знания и опыт для формирования когерентной модели текста (т.е. ситуационной модели(по Кинчу)).

Анализ текста на английском языке осуществляется следующим образом:

Параметры поверхностного кода Лексические и семантические метрики оцениваются в Coh-Metrix на основе баз данных и корпусов, включая MRC[10]. Автоматический анализатор рассчитывает многозначность слов, возраст освоения, образность, абстрактность, долю знаменательных слов в тексте и др. Следует отметить, что частота слов рассчитывается на основе данных CELEX[10], неоднозначность и иперонимический уровень на основе WordNet[10].

Синтаксическая сложность текста Синтаксическая сложность текста рассчитывается при помощи синтаксического анализатора Е.Черняк(2000)[11] на основе количества именных и глагольных групп, предложных словосочетаний, встроенных конструкций, модификаторов именных групп, слов перед сказуемым в главном предложении и др. .

Уровень текста Coh-Metrix рассчитывает следующие типы лексической кореферентности: повторы нарицательных существительных, местоимений, основ и знаменательных частей речи как в смежных предложениях, так и во всем тексте.

Глубинная связность на уровне предложений и текста рассчитывается на основе частоты встречаемости следующих типов дискурсивных маркеров: аддитивные(также, кроме того), темпоральные(а затем, после, во время), каузальные(потому что, так) и логические(следовательно, если, и, или)[4] .

Лексическое разнообразие рассчитывается как отношение неповторяющихся в тексте(от англ. types) и словоформ (от англ. tokens)

Важным условием понимания текста является способность моделирования ситуационной (или ментальной) модели, «референциального содержания или микромира того,

о чем текст»[13]. Ситуационная модель текста экиплицируется в причинно-следственных связях, интенциональности, темпоральности, пространственности и количестве субъектов коммуникативной ситуации[14], рассчитываемых при помощи следующих метрик: количество каузальных глаголов, каузальных структур (потому что, как следствие, как результат), интенциональных глаголов, интенциональных структур (чтобы, с помощью, посредством), морфологических повторов (времени и вида глагола), отношение каузальных структур к каузальным глаголам, отношение интенциональных структур к интенциональным глаголам и др. Очевидно, например, что маркеры несовпадающих временных форм затрудняют построение ситуативной модели, а присутствие темпоральных маркеров (позднее, до того, как, накануне),

наоборот, снижают сложность текста.

Пространственность как «пространственная плотность» (spatial density) и «пространственная связность» (spatial cohesion) оценивается путем подсчета доли существительных с семантикой местоположения и глаголов движения[4].

Coh-Metrix также оценивает концептуальное сходство между предложениями и абзацами при помощи латентного семантического анализа(ЛСА), статистического метода представления границ. Большой интерес вызывает проект Ивана Бегтина, под названием Plain Russian Language(Понятный/простой русский язык)[2]. Для проекта автор разработал датасет из русскоязычных текстов. Датасет состоит из экспертно размеченных взрослых текстов, текстов для внеклассного чтения, особо сложных текстов законов и других.

В проекте мы можем найти программу, в которой приводится оценка сложности текста. Центральное место в программе оценки сложности занимает функция `generate_all_metrics`. Основная ее задача - подсчет метрик на текстах из датасета, которые находятся в папке `texts`, находящейся в той же директории, что и файл с программой оценки сложности.

Отдельные текстовые единицы типа количества предложений, количество слов, количество слогов считались при помощи функции `calc_readability_metrics()`. В этой же функции происходит подсчет основных метрик удобочитаемости.

Так как целью программы являлся подсчет удобочитаемости русскоязычных текстов, автору пришлось адаптировать известные метрики удобочитаемости для русского языка. Он делал это практически вручную, задавая комбинацию коэффициентов и

выбирая наилучшую из них. То есть для констант был задан диапазон их вероятных значений с шагом 0.0001; каждая тройка констант подставлялась в метрику удобочитаемости, получалось значение. После этого высчитывалось среднее отклонение полученного значения по всем текстам. В результате из всех вариантов констант отбирались те, по которым средние отклонения минимальны.

степень точности выделения отдельных текстовых единиц достаточно низкая. Tokenization (деление на слова) и сегментация (деление на предложения) в проекте И.В.Бегтина сделаны крайне просто. При подсчёте длины слов текст разбивается по пробелам, и в каждом фрагменте подсчитывается количество русских букв. В результате склеиваются слова, написанные через тире или дефис (“поезд 7 Москва—Петербург”). Количество предложений определяется как количество точек, вопросительных и восклицательных знаков. Это приводит к нескольким неточностям: 1. точки в сокращениях считаются разделителями предложений (“См. пункт 3.” — 2 предложения); 2. группа вопросительных и восклицательных знаков обрабатывается прямолинейно (“Что?!!!” — 4 предложения); 3. в коротких текстах без конечной точки количество предложений принимается за ноль (“Пример без точки :-”) — 0 предложений), а при подсчёте средней длины слова (n_words / n_sents) происходит ошибка деления на ноль

4 Теоретическая часть

4.1 Классические метрики удобочитаемости

В настоящее время наиболее известными метриками удобочитаемости являются 5 метрик, разработанных в США для английского языка. Это индексы Флэша-Кинкэйда, Колман-Лиану, SMOG, Дэйла-Чалл и ARI.

Индекс Флэша-Кинкэйда разработан по заказу армии США для оценки удобочитаемости инструкций использования военной техники в 1975 году. Его цель - добиться уровня удобочитаемости военных инструкций для понимания учениками 9-х классов.

Это индекс удобочитаемости устроен так, что учитывает длину предложений и слов заданного текста. Значения данного индекса лежат в диапазоне от 0 до 100 единиц. Формула метрики выглядит так: $F = 206,335 - [1,015 * S + 84,6 * W]$, где S – среднее количество слов в предложении, W – среднее число слогов в слове. В большинстве случаев значения индекса F лежат в диапазоне от 0 до 100. В пределах этих значений выделяют следующие градации индекса F и соответствующие им уровни читабельности: 90-100 свидетельствуют о высокой легкости чтения текста, поэтому они легко воспринимаются младшими школьниками. $F = 80-90$ соответствует уровню шестого класса, 70-80 – седьмого, 60-70 – 8-9 классов, 50-60 и ниже – 10-11 классов, 30-40 – это уровень колледжа. Индекс $F = 0-30$ соответствует очень низкому уровню читабельности и означает, что тексты с такими индексами требуют высокого уровня образованности для понимания. Такие значения индекса имеют, как правило, научные и специальные тексты.

Теперь рассмотрим тест Колман-Лиану[15]. Этот тест был разработан Мэри Колман и Т.Л.Лиану. В этом тесте в качестве характеристики для подсчета используется количество букв. Также интересным является то, что в этой формуле учитывается среднее число букв на слово и среднее число слов на предложение, таких параметров нет ни в одном другом индексе удобочитаемости. Формула этого индекса выглядит следующим образом: $CLI = 0.0588 * L - 0.296 * S - 15.8$

Интерес также представляет формула SMOG, разработанная Harry McLaughlin в 1969 году и опубликованная в работе «SMOG Grading — a New Readability Formula»[16] В данной формуле учитываются сложные слова. Слово в данном индексе считается сложным, если в нем количество слогов превышает определенное значение. В формуле производится оценка пропорции сложных слов к числу предложений.

Формула представляет из себя выражение:

$$grade = 1.0430 * \sqrt{difficult_words} * \frac{30}{sentences} + 3.1291$$

В итоговой формуле индекса SMOG считается сложным слово с 3-мя и более слогами.

Наибольшее применение формула SMOG получила при оценке сложности медицинских текстов и в последующих исследованиях она показала большую точность по сравнению с формулой Флэша-Кинкэйда.

Формула Дэйла-Чалл[17] была разработана Эдгаром Дэйлом и Джоан Чалл в 1948 году из списка 763 слов. Сложные слова определялись следующим образом: простыми считались те слова, с которыми были знакомы 80 процентов учеников 4-х классов, к сложным словам, соответственно, относились оставшиеся. Вскоре появилась обновленная формула, в которой список узнаваемых слов был расширен - в него входило 3000 языковых единиц.

Формула Дэйла-Чалл представляет из себя следующее выражение: $0.1579(\frac{difficult_words}{words} * 100) + 0.0496(\frac{words}{sentences})$. Эта формула использовалась по большей части для проверки знаний школьников, которые учились в классах старше четвертого.

Последней метрикой удобочитаемости является ARI. Она была опубликована в 1967 году и учитывает число букв, для того чтобы поставить правильный класс образования при оценке сложности текста. Формула представляет из себя выражение вида:

$$grade = \alpha * \frac{difficult_words}{words} + \beta * \frac{words}{sentences}$$

4.2 Методы предварительной обработки данных

4.2.1 Стандартная нормализация признаков

Требование нормализации значений признаков является общим требованием для многих моделей машинного обучения. Некоторые модели могут вести себя плохо, если распределение признаков не соответствует нормальному. В связи с этим применяется такой вид нормализации признаков, как StandardScaler.

Класс `sklearn.preprocessing.StandardScaler` библиотеки `Sklearn` реализует метод приведения признаков к нулевому среднему и единичной дисперсии. Класс принимает значение таких параметров, как `copy`, `with_mean` и `with_std`. Данный вид нормализации изменяет признаки путем вычитания среднего из каждого признака и деления его ве-

личины на величину стандартного отклонения. Значения величины стандартного отклонения и среднего собираются по всей выборке.

После применения данного метода значения каждого признака имеют вид: $z = (x - u)/s$, где u - среднее значение признаков в обучающей выборке, и s - стандартное отклонение признаков.

4.2.2 Нормализация приведением к диапазону

Метод приведения признаков к заданному диапазону реализует класс `sklearn.preprocessing.MinMaxScaler`. Данный класс принимает параметр `feature_range`, обозначающий диапазон значений, в который должен попасть признак.

Признаки преобразуются следующим образом:

$$X_{std} = (X - X_{min}) / (X_{max} - X_{min}); X_{scaled} = X_{std} * (max - min) + min$$

Данный вид нормализации часто используется как альтернатива масштабированию признаков по методу `StandardScaler`.

Метод `MinMaxScaler` не уменьшает влияние выбросов, он линейно масштабирует их до заданного диапазона, где наибольшая встречающаяся точка данных соответствует максимальному значению, а наименьшая встречающаяся точка соответствует минимальному значению.

4.3 Критерии отбора признаков

4.3.1 Критерий отбора признаков по величине коэффициента в формуле линейной регрессии

Данный способ отбора признаков подразумевает обучение модели линейной регрессии. Формула данного метода представляет из себя выражение вида: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$, где X_1, X_2, X_3 - отдельные характеристики в тексте, $\beta_0, \beta_1, \beta_2$ - коэффициенты в формуле линейной регрессии. Для проведения отбора требуется обучить модель на том же наборе данных, на котором будет производиться обучение модели, для которой проводится отбор признаков. После этого оценивается значение модуля коэффициента при каждом признаке - чем выше значение модуля в итоговой формуле, тем выше значимость данного признака для модели.

4.3.2 Отбор признаков по коэффициенту корреляции

В статистике коэффициент корреляции является мерой криволинейной зависимости между статистической дисперсией внутри отдельных категорий и дисперсией по всей совокупности или выборке. Мера определяется как соотношение двух стандартных отклонений, представляющих эти типы вариаций. Контекст здесь тот же, что и у коэффициента внутриклассовой корреляции, значением которого является квадрат коэффициента корреляции.

Основные свойства η :

- η характеризует степень тесноты любой корреляционной связи (как линейной, так и нелинейной), в отличие от коэффициента корреляции Пирсона r , который характеризует тесноту только линейной связи. Условие $r = 0$ означает отсутствие линейной корреляционной связи между величинами, но при этом между ними может существовать нелинейная корреляционная связь $\eta \geq 0$.
- η принимает значения от 0 до 1; при $\eta = 0$ корреляционная связь отсутствует, при $\eta = 1$ связь считается функциональной; степень тесноты связи можно оценивать по различным общепринятым шкалам, например, по шкале Чеддока и др.
- Величина η^2 характеризует долю вариации, объясненной корреляционной связью между рассматриваемыми переменными.
- η не может быть меньше абсолютной величины r : $\eta \geq |r|$

η несимметрично по отношению к исследуемым переменным, то есть $\eta_{XY} \neq \eta_{YX}$

Для расчета η необходимо иметь эмпирические данные эксперимента с повторениями; если же мы имеем просто два набора значений переменных X и Y , то данные нужно группировать. Этот вывод, в общем-то, очевиден - если предпринять попытку рассчитать по негруппированным данным, получим результат $\eta = 1$.

Группировка данных для расчета заключается в разбиении области значений переменных X и Y на интервалы, подсчет частот попадания данных в интервалы и формирование корреляционной таблицы.

4.4 Обзор алгоритмов многоклассовой классификации

4.4.1 Метод k ближайших соседей

Один из наиболее распространенных алгоритмов классификации — это метод k ближайших соседей, который используется для классификации объектов на основе их близости к другим объектам в обучающей выборке.

Алгоритм k ближайших соседей основан на принципе близости объектов в пространстве признаков. Он состоит в следующем: объект из тестовой выборки относится к классу k его ближайших соседей из обучающей выборки. Класс, который наиболее часто встречается среди соседей, и будет классом, к которому относится исходный объект.

Реализацию данного метода можно найти в библиотеке `sklearn`.

Преимущества алгоритма k ближайших соседей в задачах классификации заключаются в его простоте и интуитивности. Однако, он может быть неэффективен в случаях, когда обучающая выборка имеет большое количество атрибутов или объектов, поскольку поиск ближайших соседей может быть очень ресурсоемким.

Кроме того, выбор оптимального значения k может оказаться нетривиальной задачей. Слишком маленькое значение k может привести к переобучению модели, тогда как слишком большое значение k может привести к недообучению модели. Для решения данной проблемы используется кросс-валидация.

Перед проведением многоклассовой классификации методом ближайшего соседа требуется провести нормализацию обучающих данных, так как признаки могут иметь разную физическую природу и, соответственно, иметь разные диапазоны значений.

4.4.2 Деревья решений

Деревья решений - это метод обучения с учителем, используемый в задаче классификации и регрессии. Идея метода заключается в том, чтобы создать модель, которая предсказывает значение целевой переменной, используя простейшие методы принятия решений. Результаты, полученные с помощью метода решающих деревьев, представляются как кусочно постоянная аппроксимация.

Некоторые преимущества алгоритма деревьев решений:

- Алгоритм легко понять и интерпретировать

- Деревья можно визуализировать
- Для данного метода не требуется длительной подготовки данных. Другие методы машинного обучения требуют нормализации данных, создания dummy значений, удаления нулевых значений, в то время как данный метод этого не требует
- Длительность предсказания имеет логарифмическую зависимость от числа точек, используемых при обучении дерева
- Данный метод поддерживает использование как числовых, так и категориальных переменных
- Данный метод является методом, использующим стратегию белого ящика. Если мы получаем некоторые результаты их легко интерпретировать, используя Булеву логику. В то время как модели, использующие стратегию черного ящика дают результаты, которые сложно интерпретировать.
- Модель поддается валидации с использованием статистических тестов. Этот способ позволяет контролировать надежность модели.
- Данный метод работает хорошо даже если его предположения каким-то образом нарушаются первоначальной моделью, на которой были получены данные.

К недостаткам модели решающих деревьев относятся:

- Деревья решений могут создавать слишком сложные деревья, которые плохо генерируют выборку. Обычно, чтобы избежать этого используют такие механизмы, как прунинг. Также, чтобы избежать данной проблемы можно установить минимальное количество объектов, требуемых для узла дерева или установить максимальную глубину дерева.
- Решающие деревья являются нестабильными - даже небольшие изменения в выборке могут привести к построению совершенно другого дерева. Данной проблемы можно избежать применяя данный метод в ансамбле моделей.
- Для данного метода не требуется длительной подготовки данных. Другие методы машинного обучения требуют нормализации данных, создания dummy значений, удаления нулевых значений, в то время как данный метод этого не требует

- Длительность предсказания имеет логарифмическую зависимость от числа точек, используемых при обучении дерева
- Данный метод поддерживает использование как числовых, так и категориальных переменных
- Данный метод является методом, использующим стратегию белого ящика. Если мы получаем некоторые результаты, как выход модели, то их просто интерпретировать, используя Булеву логику. В то как модели, использующие стратегию черного ящика дают результаты, которые сложно интерпретировать.
- Модель поддается валидации с использованием статистических тестов. Этот способ позволяет контролировать надежность модели
- Данный метод работает хорошо даже если его предположения каким-то образом нарушаются первоначальной моделью, на которой были получены данные
- Существуют задачи, которые не решаются данным алгоритмом. к этим задачам относятся задача моделирования поведения функции XOR, задача определения четности числа, а также построение функции, моделирующей мультиплексор
- Деревья решений могут создавать неправильные деревья, если некоторые классы доминируют, поэтому требуется сбалансировать набор данных перед использованием метода

Деревья решений — это один из наиболее простых и интерпретируемых алгоритмов машинного обучения, который может быть использован для решения различных задач, например, прогнозирования кредитного скоринга, диагностики заболеваний, определения температуры на улице и т.д.

4.4.3 Наивный байесовский классификатор

Наивный байесовский классификатор (Naive Bayes classifier) — вероятностный классификатор на основе формулы Байеса со строгим (наивным) предположением о независимости признаков между собой при заданном классе, что сильно упрощает задачу классификации из-за оценки одномерных вероятностных плотностей вместо одной многомерной.

В данном случае, одномерная вероятностная плотность — это оценка вероятности каждого признака отдельно при условии их независимости, а многомерная — оценка вероятности комбинации всех признаков, что вытекает из случая их зависимости. Именно по этой причине данный классификатор называется наивным, поскольку позволяет сильно упростить вычисления и повысить эффективность алгоритма. Однако такое предположение не всегда является верным на практике и в ряде случаев может привести к значительному ухудшению качества прогнозов.

Сама же формула Байеса выглядит следующим образом:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)},$$

где:

- $P(A|B)$ — апостериорная вероятность события A при условии выполнения события B
- $P(B|A)$ — условная вероятность события B при условии выполнения события A
- $P(A)$ и $P(B)$ — априорные вероятности событий A и B соответственно

В контексте машинного обучения формула Байеса будет иметь следующий вид:

$$P(y_k|X) = \frac{P(y_k)P(X|y_k)}{P(X)},$$

где:

- $P(y_k|X)$ — апостериорная вероятность принадлежности образца к классу y_k с учётом его признаков X
- $P(X|y_k)$ — правдоподобие, то есть вероятность признаков X при заданном классе y_k
- $P(y_k)$ — априорная вероятность принадлежности случайно выбранного наблюдения к классу y_k
- $P(X)$ — априорная вероятность признаков X

Если объект описывается не одним, а несколькими признаками $X_1, X_2, \dots, X_n$, то формула принимает вид:

$P(y_k | X_1, X_2, \dots, X_n) = \frac{P(y_k) \prod_{i=1}^n P(X_i | y_k)}{P(X_1, X_2, \dots, X_n)}$ На практике числитель данной формулы представляет наибольший интерес, поскольку знаменатель зависит только от признаков, а не от класса, и поэтому часто он опускается при сравнении вероятностей разных классов. В конечном счёте правило классификации будет пропорционально выбору класса с максимальной апостериорной вероятностью:

$$y_k \propto \arg \max_{y_k} P(y_k) \prod_{i=1}^n P(X_i | y_k)$$

Гауссовский наивный байесовский классификатор (GaussianNB) — вариант для работы с непрерывными признаками, которые имеют нормальное (гауссовское) распределение. Вероятность признака при заданном классе вычисляется по формуле:

$P(x_i | y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$ где μ_y и σ_y — это среднее и стандартное отклонения признака в классе y . Эти параметры оцениваются с помощью метода максимального правдоподобия по обучающим данным.

Алгоритм работы наивного байесовского классификатора строится следующим образом:

1. Изначально рассчитываются априорные вероятности классов
2. После рассчитываются средние и стандартные отклонения признаков по классам
3. На основе полученных отклонений признаков по классам рассчитывается вероятностная плотность тестовых признаков по распределению Гаусса
4. Далее вычисляются апостериорные вероятности как произведение априорных вероятностей классов и вероятностных плотностей тестовых признаков
5. Классы с максимальной апостериорной вероятностью будут итоговым прогнозом

4.4.4 Логистическая регрессия

Логистическая регрессия является частным случаем линейного классификатора, но она обладает хорошим "умением" — прогнозировать вероятность p_+ отнесения примера $\vec{x}_i$ к классу "+":

$$p_+ = P(y_i = 1 | \vec{x}_i, \vec{w})$$

Прогнозирование не просто ответа ("1" или 1), а именно вероятности отнесения к классу "+" во многих задачах является очень важным бизнес-требованием. Например,

в задаче кредитного скоринга, где традиционно применяется логистическая регрессия, часто прогнозируют вероятность невозврата кредита (p_+). Клиентов, обратившихся за кредитом, сортируют по этой предсказанной вероятности (по убыванию), и получается скоркарта — по сути, рейтинг клиентов от плохих к хорошим.

Итак, логистическая регрессия прогнозирует вероятность отнесения примера к классу "+" (при условии, что мы знаем его признаки и веса модели) как сигмоид-преобразование линейной комбинации вектора весов модели и вектора признаков примера:

$$p_+(x_i) = P(y_i = 1 \mid \vec{x}_i, \vec{w}) = \sigma(\vec{w}^T \vec{x}_i).$$

4.5 Метрики оценки качества

4.5.1 Метрика ассигасу

Ассигасу — это показатель, который описывает общую точность предсказания модели по всем классам. Это особенно полезно, когда каждый класс одинаково важен. Он рассчитывается как отношение количества правильных прогнозов к их общему количеству.

$$Accuracy(y, y^{pred}) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y_i = f(x_i)], \text{ где:}$$

N — общее количество примеров в выборке.

y_i — истинное значение для (i)-го примера.

$f(x_i)$ — предсказанное значение для (i)-го примера.

$\mathbb{I}[\cdot]$ — индикаторная функция, которая равна 1, если условие внутри скобок истинно, и 0 в противном случае.

Для той же метрики Ассигасу, ее сопряженная метрика — доля ошибочных классификаций, или Error Rate, определяемая следующим образом:

$$ErrorRate(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y_i \neq f(x_i)]$$

Преимущества метрики Ассигасу:

Простота интерпретации: Ассигасу является интуитивно понятной метрикой, которую легко интерпретировать. Она предоставляет ясное представление о том, насколько модель правильно классифицирует данные, что упрощает оценку ее общей производительности. Универсальность: Метрика Ассигасу может быть применена в различных областях машинного обучения, включая задачи классификации, регрессии и кластеризации. Это делает ее удобной для сравнения производительности различных моделей в

разных контекстах.

Универсальность: Метрика Ассигасу может быть применена в различных областях машинного обучения, включая задачи классификации, регрессии и кластеризации. Это делает ее удобной для сравнения производительности различных моделей в разных контекстах.

Ограничения метрики Ассигасу:

Неравномерные классы: В случае, если классы в данных несбалансированы, Ассигасу может быть искажена, поскольку она не учитывает дисбаланс между классами. Где класс, который имеет метку “1”, определяется как положительный класс, в то время как класс с меткой “0” называется отрицательным классом.

Например, в задаче диагностики редких заболеваний классификатор, предсказывающий всем пациентам отсутствие болезни будет иметь достаточно высокую ассигасу просто потому, что больных людей в выборке намного меньше.

Игнорирование ошибок: Ассигасу не различает типы ошибок, совершаемых моделью. В некоторых задачах, где определенные ошибки имеют более серьезные последствия, такое игнорирование может быть нежелательным. Для примера снова можно привести задачу медицинской диагностики: если ошибочный положительный диагноз для здорового больного обернется лишь ещё одним обследованием, то ошибочно отрицательный вердикт может повлечь роковые последствия.

5 Практическая часть

5.1 Предложенный подход к оценке удобочитаемости

Идея предложенного подхода состоит в том, что на первом этапе выбирается совокупность алгоритмов многоклассовой классификации и достаточно большое множество параметров текста. Для этого был проведен анализ известных алгоритмов и существующих подходов и работ в области оценки удобочитаемости. Результаты этого анализа частично приведены в 3 и 4 главах настоящей работы. На втором этапе проводится Использование алгоритмы, методы нормализации и отбора параметров описаны в главе 4. Таким образом, было получено множество моделей классификации, с каждой из которых были проведены эксперименты, описанные ниже в п.5.5.

Для обучения выбранных моделей был собран датасет размером 166 текстов, преимущественно состоящий из методических материалов для средней школы. Этот датасет описан ниже - в п. 5.1.2. Автоматическая обработка текста и подсчет параметров (основных характеристик) текста проводились с помощью программного кода на языке Python.

Перед настройкой модели требовалось сформировать обучающий и тестовый наборы данных. Эти наборы данных представляют из себя матрицы размерности (166, 40000), 166 - по числу текстов в датасете и 40000 по числу признаков. Признаки брались из множества, которое включало в себя такие признаки как количество слов, обладающих конкретной граммемой, количество слов, входящих в один из словарей частотности, количество слов с определенной концовкой словоформы, также подсчитывались значения таких показателей, как количество предложений в тексте, количество слогов, количество сложных слов. Подход к формированию таких типов признаков был заимствован в работе [3].

Перед обучением и оценкой качества модели требовалось отобрать признаки и нормализовать данные. Для этих целей использовались такие подходы к нормализации данных как MinMaxScaler и StandardScaler (см. главу 4). В качестве подходов к отбору признаков применялся отбор по максимальным модулям коэффициентов линейной регрессии (п.4.3.1) и отбор по значимости признака по величине коэффициента корреляции (п.4.3.2).

В число отобранных алгоритмов классификации вошли алгоритм решающих дере-

вьев, метод k ближайших соседей, наивный байесовский классификатор и логистическая регрессия с применением подхода ovr ("один против остальных"). Эти алгоритмы описаны в главе 4. Отметим, что никакие нейросетевые алгоритмы не использовались, хотя подходы, использующие нейронные сети, доказали свою эффективность во многих задачах обработки текста. Но для достижения высокого качества обучения нейросетей требуются датасеты, значительно превосходящие по объему созданный датасет.

Для оценки качества обученных моделей использовалась метрика ассигасу.

5.2 Разработка датасета

Датасет состоит из текстов, разделенных по 4 классам сложности. К первому классу сложности отнесены тексты, составленные для начальных классов. Ко второму классу отнесены тексты для средних классов школы. К третьему классу отнесены тексты, составленные для старших классов, последний класс сложности включает в себя тексты учебников для ВУЗов, а также особо сложные тексты законов, тексты новостных статей, тексты из Википедии.

Первоначально тексты находились в форматах pdf, docx и pptx. Чтобы получить отдельный текст в формате txt, требовалось применять конвертеры.

Тексты, относящиеся к первому, второму и третьему классу сложности были получены из презентаций, выложенных на интернет сайтах, разработанных для педагогов. Каждая презентация скачивалась и переводилась при помощи конвертера в формат txt. При разработке конвертера для перевода текстов презентаций применялись средства библиотеки pptx.

Тексты, относящиеся к 4 классу сложности, преимущественно были представлены в формате pdf. Для получения обычного формата текста (plain text) использовался конвертер из библиотеки PDFReader.

Также в датасет добавлены 4 текста для внеклассного чтения, которые изначально находились в форматах docx. Для перевода этих текстов использовался конвертер из библиотеки ruandoc, переводящий тексты из формата docx в формат txt.

Помимо файлов, переведенных в формат txt при помощи конвертеров, в датасет добавлены тексты из датасета проекта Plain Russian Language. Этот датасет имеет следующую структуру: первые 3 папки содержат тексты произведений школьной программы, к ним относятся тексты таких произведений, как "Приключения Буратино текст

сказки Курочка Ряба; для третьего класса добавлены тексты произведения "Белый Бим Черное Ухо" Стального кольца Паустовского, "Горячий камень" Аркадия Гайдара. Номер папки соответствует классу сложности текстов из папки. В четвертый класс сложности добавлены тексты таких произведений, как "Бронзовая птица" Рыбакова, повесть Гайдара "Судьба барабанщика" текст произведения Д. Григоровича "Гуттаперчевый мальчик" текст повести А.Н. Толстого "Детство Никиты". Таким образом, для первых четырех классов добавлены тексты произведений начальной школы. В папке с номером 5 находится текст произведения "Алиса в стране чудес". Папка с номером 6 содержит текст произведения Солоухина "Мститель" и рассказ "Срезал" В.М. Шукшина. В папке с номером 10 и 11 представлены тексты, которые проходят в старшей школе. В папке с номером 12 находится текст из Википедии. В папке с номером 15 содержится текст из "Российской газеты" комментарий к изменениям в подразделе 3 раздела I части первой ГК РФ. В папке с номером 17 находятся тексты с протоколом заседания правительства, проведенном в 2013 году. При переносе текстов в собранный датасет потребовалось распределить их по 4 классам сложности. Тексты из первых 4 папок датасета И.В. Бегтина были добавлены в папку собранного датасета с номером 1, тексты из папок Бегтина с номерами от 5 до 9 были добавлены в папку с номером 2, тексты из папок с номерами 10, 11 были добавлены в папку с номером 3, оставшиеся тексты добавлялись в папку с номером 4. Финальным этапом в разработке датасета стало добавление текстов диктантов для средней школы. Общее количество текстов после их добавления составило 166.

5.3 Подсчет точности формул И.В.Бегтина

Результаты полученных в работе моделей сравнивались с результатами формул из проекта Plain Russian Language И.В.Бегтина. В программе оценки удобочитаемости этого проекта реализован подсчет 10 метрик. Каждая из метрик считает какую-то классическую метрику удобочитаемости из перечисленных в п. 4.1 выше. В программе реализован подсчет следующих метрик:

- `grade_SMOG` - возвращает уровень образования необходимый для понимания текста по формуле SMOG
- `grade_ar1` - возвращает уровень образования необходимый для понимания текста

по формуле Automated Readability Index

- `grade_cl` - возвращает уровень образования необходимый для понимания текста по формуле Coleman-Liau
- `grade_fk` - возвращает уровень образования необходимый для понимания текста по формуле Flesch-Kincaid
- `grade_dc` - возвращает уровень образования необходимый для понимания текста по формуле Dale-Chale

Каждая из метрик возвращает уровень образования, необходимый для понимания текста по российской градации школьного образования,

Для подсчета точности формул, разработанных Бегтиным, каждому тексту сопоставлялся вектор значений с основными характеристиками текста, требуемыми для получения значения по формуле.

Эти вектора формировали информационную матрицу.

Далее с помощью этой матрицы, осуществлялся подсчет значений формулы. Величина значений формулы могла находиться в диапазоне от минус бесконечности до бесконечности. Так как подсчет значений точности осуществлялся на текстах из датасета, ранжированных от 1 до 4, то требовалось задать пороги для перевода классов Бегтина в четыре класса насоящей работы. Эти пороги выбраны так: тексты со значениями от минус бесконечности до четырех относились к первому классу сложности, тексты с пятого по восьмой класс сложности относились ко второму классу, тексты с восьмого по двенадцатый класс относились к третьему классу и, наконец, оставшиеся относились к 4 классу сложности.

В итоге получились следующие показатели метрики ассигасу:

	Метрика удобочитаемости	accuracy score
1	Метрика Flesh Kincaid rus	0.157
2	Метрика Flesh Kincaid grade rus	0.315
3	Метрика Flesh Kinkcaid grade rus flex	0.218
4	Метрика SMOG index	0.127
5	Метрика Dale Chale index	0.175
6	Метрика ARI index	0.272

Таблица 1: Точность формул И.В.Бегтина

5.4 Подготовка данных для обучения и оценки качества моделей

Данные для обучения и оценки качества моделей представлены двумя матрицами. Первая из них - информационная матрица размерности (166, 40000), 166 - по числу текстов в датасете, 40000 по числу признаков. Для формирования этой матрицы используется функция построения признакового описания текста. Эта функция формирует вектор признаков для каждого текста из датасета. Подсчитываются значения таких признаков, как количество предложений, количество слогов, количество слов, обладающих заданной концовкой словоформы, количество слов, обладающих конкретной граммемой, количество слов, входящих в один из словарей частотности. Словари частотности брались из ресурса[5]. Вторая матрица - это матрица со значениями целевой переменной, то есть метками классов сложности, к которым принадлежали тексты. Размерность данной матрицы соответствует числу текстов в датасете.

5.5 Эксперименты

Общее количество экспериментов составило 1600. Как уже говорилось выше каждый эксперимент отличается выбором алгоритма, а именно: деревья решений, k ближайших соседей, наивный байесовский классификатор, логистическая регрессия с применением подхода "один против остальных"; способом нормализации данных: используемые под-

ходы: StandardScaler, MinMaxScaler, способом отбора признаков: по значению коэффициента в формуле линейной регрессии, значением величины коэффициента корреляции; количеством признаков, используемых в модели. Показатель данной величины менялся от 1 до 100.

Для получения этих признаков предварительно формируется вектор из 40000 признаками, упорядоченных по одному из двух способов отбора признаков. Далее последовательно проверяется качество моделей с первым признаком из вектора, с первыми двумя признаками и так далее вплоть до 100 признаков.

Для каждого эксперимента построен график, в котором на оси x откладывается количество использованных признаков, а на оси y - показатель качества по метрике ассигасу.

Интерес с точки зрения зависимостей на графиках представляют эксперименты с номерами 1 и 2. В этих экспериментах применяется один и тот же алгоритм логистической регрессии. Однако зависимости на графиках имеют совершенно разных характер. На графике первого эксперимента кривая содержит много изломов, в то время как кривая второго эксперимента, в котором применялся MinMaxScaler в качестве метода нормализации, вела себя более устойчиво. Таким образом, можно сделать вывод, что способ нормализации может серьезно влиять на поведение моделей.

Для кривых, построенных по методу решающих деревьев в экспериментах 8 и 10 характерно множество изломов. То есть здесь снова наблюдается что изменение количества признаков сильно влияет на качество модели.

Если говорить о значении метрики ассигасу, то можно заметить, что в экспериментах с номерами 3 и 4, 5 и 6, 7 и 8, 15 и 16 смена способа нормализации данных не повлияла на величину показателя максимального значения ассигасу. В то же время, если посмотреть на итоговые значения метрики ассигасу в экспериментах 3 и 4, 9 и 10, 11 и 12, 13 и 14, то можно увидеть, что StandardScaler работает лучше, чем MinMaxScaler.

Также, если посмотреть на таблицу экспериментов, можно понять, что классификация методом решающих деревьев дает большие показатели точности, чем прочие виды методов классификации. После этого подхода следующим по величине показателя ассигасу идет классификация методом ближайших соседей, после этого идет классификация по методу LogisticRegression и последним по величине показателя точности в списке оказывается классификация методом Наивного Байесовского классификатора.

В целом, при проведении экспериментов это все зависимости, которые можно было наблюдать.

Номер эксперимента	Метод предобработки данных	Критерий отбора признаков	Алгоритм	Количество признаков	accuracy
1	StandardScaler	Линейная регрессия	Logistic Regression	85	0.509
2	MinMaxScaler	Линейная регрессия	Logistic Regression	91	0.581
3	StandardScaler	Линейная регрессия	Gaussian NB	90	0.345
4	MinMaxScaler	Линейная регрессия	GaussianNB	90	0.345
5	MinMaxScaler	Линейная регрессия	KNeighborsClassifier	31	0.472
6	StandardScaler	Линейная регрессия	KNeighborsClassifier	32	0.472
7	StandardScaler	Линейная регрессия	DecisionTreeClassifier	60	0.545
8	MinMaxScaler	Линейная регрессия	DecisionTreeClassifier	46	0.545
9	StandardScaler	correlation_ratio	Logistic Regression	72	0.472
10	MinMaxScaler	correlation_ratio	Logistic Regression	90	0.436
11	MinMaxScaler	correlation_ratio	KNeighborsClassifier	59	0.618
12	StandardScaler	correlation_ratio	KNeighborsClassifier	59	0.636
13	StandardScaler	correlation_ratio	DecisionTreeClassifier	12	0.690
14	MinMaxScaler	correlation_ratio	DecisionTreeClassifier	97	0.654
15	StandardScaler	correlation_ratio	GaussianNB	10	0.527
16	MinMaxScaler	correlation_ratio	GaussianNB	10	0.527

Таблица 2: Эксперименты

StandardScaler. Линейная Регрессия. Logistic Regression

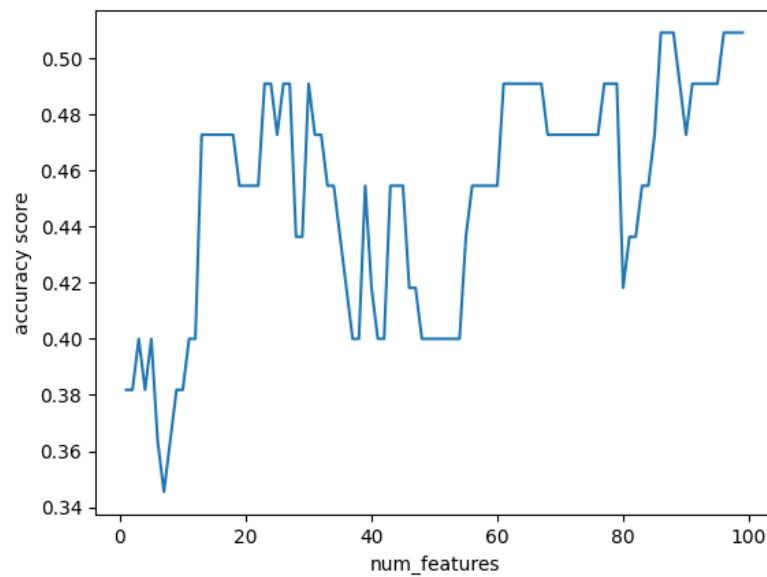


Рис. 1: Эксперимент 1

MinMaxScaler. Линейная регрессия. Logistic Regression.

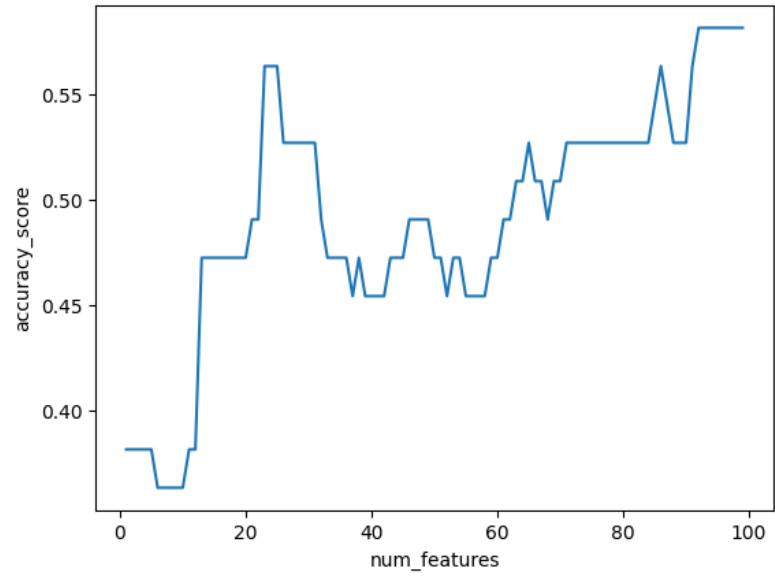


Рис. 2: Эксперимент 2

MinMaxScaler. Линейная регрессия. DecisionTreeClassifier.

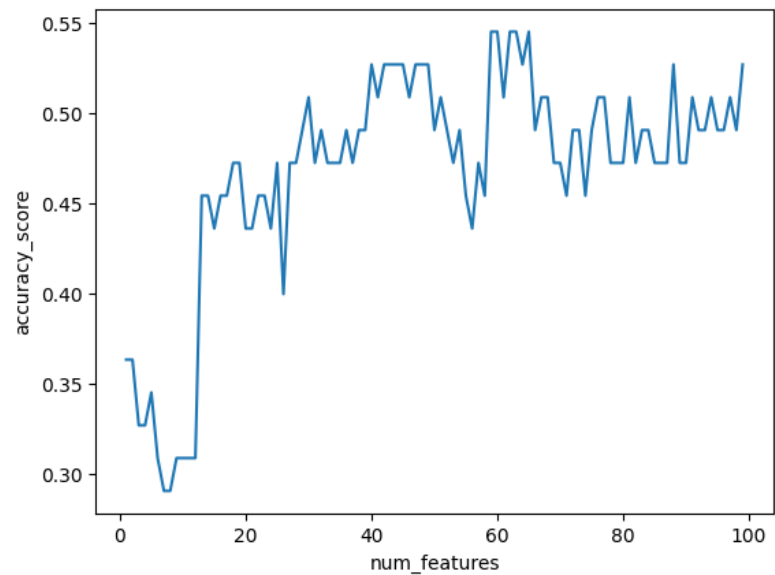


Рис. 3: Эксперимент 8

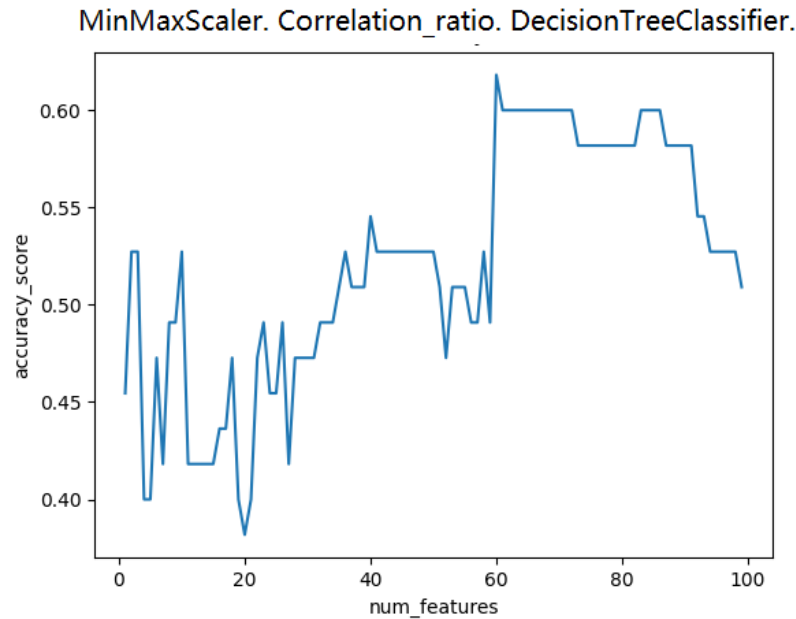


Рис. 4: Эксперимент 10

5.6 Программа автоматической оценки удобочитаемости

На основе проведенного анализа моделей классификации и экспериментов с описанным выше датасетом была разработана программа для автоматической оценки удобочитаемости.

Наилучшей моделью оказалась модель с алгоритмом классификации по методу решающих деревьев, способом нормализации StandardScaler, отбором признаков по значению величины correlation_ratio, количеством признаков равным 12.

Данная модель использовалась для подсчета сложности в программе оценки удобочитаемости.

Для того чтобы программу можно было использовать в различного рода автоматических пайплайнах, было принято решение не использовать графический интерфейс. Программа использует только интерфейс командной строки. В аргументах командной строки указывается источник ввода текста. Текст может вводиться либо из сетевого ресурса, в этом случае второй аргумент - это URL этого ресурса, либо текст берется из файловой системы, в этом случае аргумент - это путь (полный или относительный) к файлу в файловой системе. В случае, если пользователь хочет вводить текст из интер-

нет сайта, он должен задать значение первому аргументу, в данном случае это значение должно быть равно 'URL'.

Программа вводит текст полностью и подсчитывает его сложность следующим образом.

На вход предварительно обученной модели с наилучшими параметрами передается текст, модель подсчитывает уровень его сложности и возвращает в поток вывода метку класса сложности, к которому принадлежит текст.

Интерес представляет процесс извлечение текста с веб-страницы. Данный процесс проводился с помощью средств библиотеки BeautifulSoup. Дело в том, что веб-страница представляет из себя набор различных элементов, многие из которых не имеют отношение к тексту страницы, уровень сложности которого требуется подсчитать. К этим элементам, например, относится, заголовок страницы. Заголовок страницы - это большая полоса вверху страницы, с крупным заголовком или логотипом. Здесь указывается общая информация о веб-сайте, не меняющаяся от странице к странице. К элементам, содержание которых не меняется от странице к странице также относится навигационное меню - боковая панель, в содержание которой включена второстепенная информация, ссылки, цитаты, реклама и т.д. Есть и другие элементы веб-страницы, не представляющие интерес с точки зрения оценки удобочитаемости. К этим элементам относится, например, нижний колонтитул. Это полоса в нижней части страницы, которая обычно содержит уведомления об авторских правах или контактную информацию. Нижний колонтитул - это место для размещения общей информации (например, заголовка), но обычно это информация не является критичной или вторична для самого веб-сайта. Нижний колонтитул также иногда используется для SEO целей, предоставляя ссылки для быстрого доступа к популярному контенту. Навигационное меню включает в себя ссылки на основные разделы сайта; обычно в виде кнопок, ссылок или вкладок. Этот элемент веб-страницы считается частью заголовка, а не отдельным компонентом, но это не является обязательным требованием, однако, разделение данных структур на отдельные компоненты улучшает доступность, поскольку раздельная структура будет понятнее для людей, пользующихся считывателями экрана. Основной частью страницы, содержимое которой представляет интерес с точки зрения оценки удобочитаемости находится подтегом `<main>`. Этот тег помечается содержимое, уникального для этой страницы. Он размещается прямо внутри `<body>` и не должен быть вложен в другие

элементы. Для извлечения текста с веб стрницы извлекался текст данного тега и далее текст, находящийся под тегом `<p>`.

После этого данное содержимое передавалось на вход функции, которая строила вектор признакового описания текста. После этого данный вектор передавался наилучшей модели оценки удобочитаемости, которая выбиралась среди моделей, построенных в главе 5.5. Это модель и выдает уровень сложности текста, котрый выводится в поток стандартного вывода.

6 Заключение

Основные результаты работы:

1. Разработан набор данных, ранжированный по 4 классам сложности и основанный на методических материалах для средней школы
2. Проведена серия экспериментов по сравнению качества различных моделей классификации с использованием разработанного датасета
3. На основе модели, показавшей наилучшее качество из рассмотренных, разработана программа для оценки удобочитаемости текстов на русском языке

Список литературы

- [1] On the problem of text characteristics: readability, comprehensibility, complexity, difficulty / Kisel'nikov A.S. 2015 // *Philology. Theory & Practice*. — Pp. 79–84.
- [2] Begtin I.V. Plain Russian Language. URL: <https://github.com/infoculture/plainrussian>.
- [3] Readability Metrics for Russian / Druzhkin K.U. <https://www.hse.ru/en/edu/vkr/184791276>.
- [4] The Lexile Framework for Reading. URL: <https://lexile.com>.
- [5] Natural language processing and discourse complexity studies / Solnyshkina, Danielle McNamara, Radif Zamaletdinov // *Russian Journal of Linguistics*. — 2022. — no. 26(2). — P. 317–341. <https://doi.org/10.22363/2687-0088-30171>.
- [6] Ольга Ляшевская. Новый частотный словарь русской лексики. URL: <http://dict.ruslang.ru/freq.php>.
- [7] Retrieval of reading materials for vocabulary and reading practice / Heilman, Michael, Le Zhao et al. // *In Proceedings of the Third Workshop on Innovative Use of NLP for Building Educational Applications*. — 2008. — P. 80–88. <https://doi.org/10.3115/1631836.1631846>.
- [8] The TextEvaluator tool: Helping teachers and test developers select texts for use in instruction and assessment / Irene Kostin Diane Napolitano & Michael Flor Sheehan Kathleen M. — 2014.
- [9] Robert T. Millard & Raj Duvvuri Zeno Susan // *The Educator's Word Frequency Guide. Brewster: Touchstone Applied Science Associates, Inc.* — 1995.
- [10] EuroWordNet. URL: <http://wordnetweb.princeton.edu/perl/webwn>.
- [11] A maximum-entropyinspired parser / Eugene Charniak // *In Proceedings of the 1st North American Chapter of the Association for Computational Linguistics Conference*. — 2000. — P. 132–139.

- [12] Coh Metrix: Theoretical, Technological, and Empirical Foundations / McNamara, Danielle S., Arthur C. Graesser et al. // *In Automated Evaluation of Text and Discourse with Coh-Metrix*. Cambridge: Cambridge University Press. — 2014. <https://doi.org/10.1017/CB09780511894664.006>.
- [13] Constructing inferences during narrative text comprehension / Graesser, Arthur C, Matthew Singer, Tom Trabasso. // *Psychological Review* 101. — 1994. — no. 6. — P. 371–395.
- [14] Situation models in language comprehension and memory / Rolf A. & Gabriel A. Radvansky Zwaan // *Psychological Bulletin*. — 1998. — no. 123. — P. 162–185. <https://doi.org/10.1037/0033-2909.123.2.162>.
- [15] A computer readability formula designed for machine scoring / Liao T. L. Coleman Meri // *Journal of Applied Psychology*. — 1975. — Vol. 60. — P. 283–284.
- [16] Smog-grading – a new readability formula / G. Harry McLaughlin // *Journal of Reading*. — 1969. — no. 13. — P. 639–646.
- [17] Readability Revisited: The New Dale-Chall Readability Formula / Jeanne S. & Edgar Dale Chall // *Cambridge: Brookline Books*. — 1995.